MATH3888 - Stream 1
Lab 1: Python Basics & Your First Simulation

M Wechselberger

August 7, 2026

1 Goal

The goal of this lab is to install the necessary software, learn the basics of the Python programming environment, and run your first simulation of a simple ordinary differential equation (ODE).

2 Reading Material

Keener, J., & Sneyd, J. (2025). **Mathematical Physiology** (3rd ed.). Springer.

- *Chapter 1: Biochemical Reactions,* particularly
  Section 1.1: The Law of Mass Action, and
  Section 1.2: Enzyme Kinetics.

3 Requirements

Throughout this course, a working Python environment is essential as well as tools to perform bifurcation analysis. The tools we will use are:

- Python 3.x
- Jupyter Notebooks
- PyDSTool
- Docker

3.1 Software Installation: Docker

This unit uses **PyDSTool** (docs, source) to perform dynamical analysis. To perform bifurcations, it runs **AUTO**, a well-established continuation software. Installing AUTO natively requires a Fortran toolchain and can be difficult, so we have prepared a pre-built **Docker image** containing PyDSTool + AUTO + Jupyter simplifying the setup process.
Note. If you use PyDSTool in your own work, please cite: Clewley R (2012), *PLoS Comput. Biol.* 8(8): e1002628, `https://doi.org/10.1371/journal.pcbi.1002628`.

i. **Download:** Go to Docker download page: `https://docs.docker.com/desktop/setup/install/` and download depending on your operation system (Windows, macOS or Linux).

ii. **Installation:** Follow the installation steps depending on your operating system. macOS users does not need to install rosetta.

iii. **Verification:** Open your terminal and type `docker --version`. You should see something similar to `Docker version XX.X.X, build XXX`

3.2 Setup: Docker

After verifying that docker is installed, create a directory for `MATH3888`. In this directory, create a new file named `docker-compose.yml`. The content of the file should be as,

```
services:
  jupyter-service:
    image: curuy/math3888:latest
    ports:
      - "8888:8888"
    volumes:
      # Please change the directory to where you want the notebooks to be saved to
      - <PUT IN YOUR working directory here>:/notebooks
    container_name: jupyter-lab

    command: >
      jupyter lab
      --ip=0.0.0.0
      --port=8888
      --no-browser
      --allow-root
      --NotebookApp.token='password'
      --notebook-dir=/notebooks
```

Note. You can copy this file (or download it) at `https://github.com/Curuy/MATH3888`

- Modify the configuration file to set the directory to `your_local_path:/notebooks`.
- (Optional) Change the Jupyter Lab password.
- Execute the command `docker compose up` in the directory containing the `docker-compose.yml` file.
  - **Note:** Add the `-d` flag if you want Docker to run in the background.

  The `docker compose up` command reads the provided yaml file. The initial run will pull our published image from the Docker Hub that has the required packages set up. The file also attaches a volume (bind mount) to the container, linking a folder on your device to the /notebooks directory inside the container, so that notebooks and exports can be saved between sessions.

4 Introduction to Jupyter Notebooks

We will write and run our code in Jupyter Notebooks, an interactive environment that lets you mix text, equations, and live code.

i. **Launch Jupyter Notebook:** Access the Jupyter instance at `localhost:8888` or `127.0.0.1:8888`.

ii. **Create a New Notebook:** Navigate to a folder where you want to save your work (e.g., create a new folder for this course). In the top right, click "New" and select "Python 3 (ipykernel)".

iii. **Cells:** A notebook is made of cells. You can write code in a cell and run it by clicking the "Run" button or by pressing Shift+Enter.

5 Troubleshooting

Container name . . . is already in use:
Previous container was never removed (e.g. Terminal closed or Mac restarted without docker compose down). Run `docker rm -f jupyter-lab && docker compose up`.
Port 8888 already in use:
Another program (e.g. a non-Docker Jupyter) holds the port. Stop it, or change the first number in "8888:8888" (e.g. "8889:8888") and visit `http://localhost:8889`.
Note. Ending every session with `docker compose down` avoids the "name already in use" error entirely.

6 Guided Example: Simulating an ODE

Let's solve a simple decay model:

$$\frac{dy}{dt} = -0.5y$$

with the initial condition $y(0) = 10$. Type or copy the Python code from the provided `lab1_example.ipynb` file into a cell in your Jupyter notebook and run it. The comments in the code explain what each part does. You should see a plot of an exponential decay curve

7 Exercise: Model a Simple Reaction

Task: Adapt the guided example script to model the irreversible biochemical reaction $A \xrightarrow{k} B$. The ODE for the concentration of species A is

$$\frac{d[A]}{dt} = -k[A]$$

- Assume the rate constant $k = 0.1s^{-1}$
- Assume the initial concentration of A is $[A]_0 = 50\mu M$.
- Simulate the system for 30 seconds.
- Plot the concentration of A over time. Make sure to label your axes!

MATH3888 - Stream 1
Lab 1(b): An Introduction to PyDSTool

M Wechselberger

August 7, 2026

1 Goal

The goal of this lab is to use a single, classic ecological model to understand both time-course simulation and bifurcation analysis. We will first simulate the standard logistic growth model. Then, by introducing a constant harvesting term, we will use PyDSTool's bifurcation tools to find the critical harvesting threshold that leads to population collapse, which corresponds to a saddle-node bifurcation.

2 Reading Material

Keener, J., & Sneyd, J. (2025). **Mathematical Physiology** (3rd ed.). Springer.

- *Chapter 1: Biochemical Reactions*, particularly
  Section 1.6: Appendix: Math Background (Note: The key discussion of the bifurcation diagram, bistability, and saddle-node points appears on page 41, relating to the exercise below).

Note: You are supposed to read textbook material prior to Friday's lectures & labs.

2.1 Introduction to PyDSTool

After setting up the docker container, you will have a jupyter-notebook instance that has PyDSTool and its requirements installed. This tutorial will help familiarise with the tools required for this course.

3 Part I: Guided Example - Simulating Logistic Growth

Let's simulate the standard logistic growth model:

$$\frac{dx}{dt} = rx\left(1 - \frac{x}{K}\right)$$

This represents the growth of a population x with an intrinsic growth rate r and a carrying capacity K. Have a look and run the script in `lab1_logistic.ipynb`. We first define the system, solve the system numerically and then plot.

i. **Define the System in PyDSTool:** The key steps here is to define the parameters, ODEs and initial conditions and then create the ode system using the `Generator.Vode_ODEsystem`[1] function.

ii. **Solve Numerically:** First set a time range using the `set` function on the ode system and then `compute` function to compute the trajectory

iii. **Plot the Results:** Get the points from the trajectory using the `sample` method and plot it using Matplotlib.

[1] There are other integrators available but we recommend using scipy's ode solver. More information can be found at `https://pydstool.github.io/PyDSTool/Generators.html#The_ODE_.2BAC8_DAE_integrators`

4 Part II: Exercise - Bifurcation Analysis with Harvesting

Task: Now, let's introduce a constant harvesting rate, h, to the model:

$$\frac{dx}{dt} = rx\left(1 - \frac{x}{K}\right) - h$$

We will find the critical harvesting rate, h_{crit}, by starting from a known equilibrium and continuing it in the parameter h.

i. Define the new ode system with the appropriate parameters and ODEs.

ii. Define a domain for the variable x by updating `xdomain`.

iii. Import the `phaseplane` method from `PyDSTool.Toolbox` and use the `find_fixedpoints` method to look for steady states.

iv. Define the bifurcation system by using `PyCont`.

Setting	Default Value	Description	Example
freepars	[]	Bifurcation parameters to vary during continuation.	['h']
MaxNumPoints	300	Maximum number of continuation points.	200
MaxStepSize	0.1	Upper bound on the continuation step size.	1
MinStepSize	10^{-5}	Lower bound on the continuation step size.	0.001
LocBifPoints	[]	Types of bifurcation points to detect.	'all', ['H', 'LP']
SaveEigen	false	If enabled, eigenvalues are saved at each continuation point.	true
SaveJacobian	false	If enabled, Jacobian matrices are saved at each continuation point.	true
verbosity	1	Controls the amount of diagnostic output (0 = silent, 2 = most detailed).	2
initpoint	ics	Initial point where continuation starts. If not specified, the ODE initial conditions are used.	fixed_points[0]

Table 1: Continuation parameters and their default values

v. Use the `forward` or `backward` methods to compute the bifurcations.